



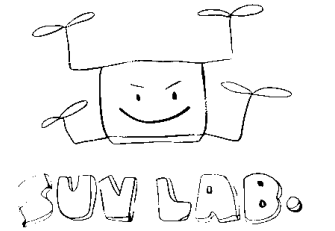
충북대학교

K-뉴딜 아카데미 · AI를 위한 PYTHON · PART 1-2

모듈과 패키지 만들기

패키지 구조부터 설치와 배포까지

충북대학교 지능로봇공학과 **문성태** 교수



무엇을 만드는가

sensorkit 패키지 제작부터 설치까지

만들 것

- **패키지** — sensorkit, 역할별 모듈로 나눈 코드
- **메타데이터** — pyproject.toml 로 이름과 버전 선언
- **테스트** — 동작을 지키는 검사 코드

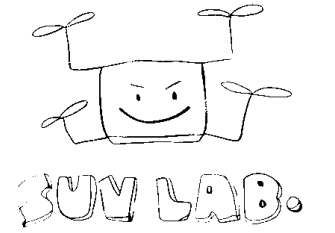
거치는 단계

- **구조** — 디렉터리와 `__init__.py`
- **공개 API** — 밖에서 쓸 이름만 노출
- **설치** — `pip install -e .` 로 어디서나 import

진행 방식

- **준비됨** — 프로젝트 파일은 이미 존재
- **보기** — 슬라이드에서 파일 내용 확인
- **작성** — 연습문제에서 빈 함수만 채워 실행

끝나면 — 다른 프로젝트에서 import 한 줄, 팀원이 같은 방법으로 설치



프로젝트 레이아웃

flat layout

- 구조 — 프로젝트 루트에 패키지 폴더를 그대로
- 장점 — 단순한 구조, 설치 없이 바로 import
- 함정 — 설치 없이도 import 되어 설치 오류를 놓침

src layout (권장)

- 구조 — src/ 아래에 패키지 배치
- 효과 — 설치해야 import, 배포본과 같은 조건에서 시험
- 부수 — tests 가 소스가 아닌 설치본을 검증

선택 — 배포용 코드는 src layout, 이 파트는 src 기준

최소 패키지 만들기

디렉터리 하나와 `__init__.py` 하나면 패키지

```
1 src/mini/__init__.py
2     __version__ = "0.1.0"
3
4 src/mini/hello.py
5     def greet(name="world"):
6         return f"hi, {name}"
```

```
1 import mini
2 from mini.hello import greet
3
4 mini.__version__, greet("sensor")
```

- **최소 조건** — 디렉터리 + `__init__.py`
- **모듈 추가** — 그 디렉터리 안에 `.py` 파일 생성
- **경로** — 상위 폴더가 `sys.path` 에 있어야 import 가능
- **확인** — 버전과 함수가 모두 보이면 성공

sensorkit 구조

이 파트에서 쓰는 프로젝트는 이미 준비되어 있음

```
1 src/sensorkit/  
2 |—— __init__.py    공개 API  
3 |—— config.py      상수  
4 |—— errors.py      예외 계층  
5 |—— io.py, clean.py, stats.py  
6 |—— report.py      요약 계산  
7 |—— cli.py         진입점  
8 |—— __main__.py    python -m 진입점  
9 |—— filters/  
10 |—— __init__.py    하위 패키지 공개 API  
11 |—— moving_average.py
```

- **io** — 로그 읽기, 열 뽑기
- **clean** — 범위 밖 값 제거
- **filters** — 신호 처리
- **stats** — 요약 통계

```
1 from pathlib import Path  
2  
3 d = Path("src/sensorkit")  
4 sorted(p.name for p in d.rglob("*.py"))
```

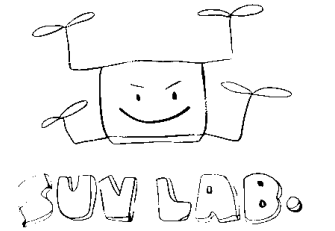

공개 API 설계

__init__.py 는 패키지의 현관

```
1 # src/sensorkit/__init__.py
2 from .clean import drop_outliers
3 from .filters import moving_average
4 from .io import column, load_csv
5
6 __all__ = ["load_csv", "column",
7           "drop_outliers", "moving_average"]
```

```
1 import sensorkit
2
3 sensorkit.__all__
4 sensorkit.moving_average([1, 2, 3, 4], 2)
```

- 재노출 — 내부 경로를 몰라도 사용 가능
- __all__ — 공개 이름 목록, 문서이자 약속
- 내부 이름 — 앞에 _ 를 붙여 비공개 처리
- 주의 — 무거운 작업은 import 지연의 원인



모듈 분리 기준

역할이 다르면 파일 분리

역할별

- **io** — 읽기와 쓰기
- **clean** — 결측과 이상치 처리
- **filters** — 신호 처리
- **stats** — 요약 통계

나누는 신호

- **길이** — 한 파일이 300줄 초과
- **이유** — 수정 이유가 둘 이상
- **의존** — 특정 함수만 무거운 패키지 사용

합치는 신호

- **동반 수정** — 항상 함께 변경
- **한 줄 모듈** — 파일 수만 증가
- **순환** — 서로를 import

순서 — 한 파일로 시작, 이유가 생겼을 때 분리

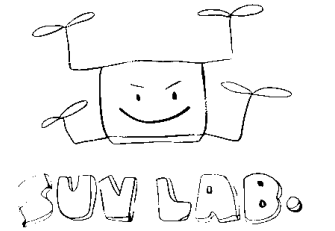
서브패키지와 상대 import

패키지 안의 패키지도 같은 규칙

```
1 # filters/__init__.py
2 from .moving_average import moving_average
3
4 __all__ = ["moving_average"]
5
6 # filters/moving_average.py
7 def moving_average(xs, n=3):
8     ...
```

```
1 from sensorkit.filters import moving_average
2
3 moving_average([1, 2, 3, 4, 5], 3)
```

- 중첩 — filters/ 도 __init__.py 보유
- 상대 import — 패키지 안에서는 .moving_average
- 절대 import — 밖에서는 sensorkit.filters
- 규칙 — 내부는 상대, 외부는 절대



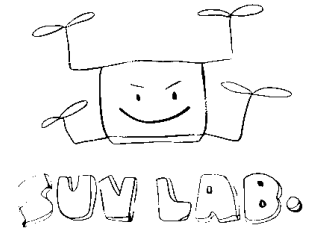
연습문제 1

왼쪽을 먼저 실행해 달려 오는 이름을 확인하고, 오른쪽에서 `__all__` 을 정의해 **summarize** 하나만 남기시오.

```
1 # 1. __all__ 이 없는 지금
2 from sensorkit import report
3
4 star_names(report)
5 # ['DIGITS', 'csv', 'mean', 'summarize']
```

```
1 # 2. __all__ 을 정의한 뒤
2 from sensorkit import report
3
4 # IMPLEMENT HERE: report.__all__ = ...
5
6 star_names(report)
7 # ['summarize']
```

report 모듈의 이름 — `csv`, `mean`, `DIGITS`, `summarize`, `_round` 다섯. `star_names` 는 모듈을 별표 import 해 실제 공개된 이름 목록을 돌려준다 . `_round` 는 밑줄로 시작해 `__all__` 없이도 빠진다



docstring과 타입 힌트

함수의 계약을 코드 안에 기록

```
1 # src/sensorkit/stats.py
2 def mean(xs: list[float]) -> float:
3     """Return the arithmetic mean of xs."""
4     return sum(xs) / len(xs)
```

```
1 from sensorkit import stats
2
3 help(stats.mean)
```

- **docstring** — 첫 줄은 한 문장 요약
- **타입 힌트** — 입력과 출력 형태를 선언
- **도구** — help(), 자동완성, 타입 검사기가 참조
- **주석과 차이** — docstring 은 실행 시에도 유지

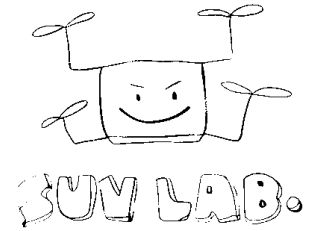
설정과 상수 관리

흩어진 값은 고칠 수 없음

```
1 # src/sensorkit/config.py
2 G_M_PER_S2 = 9.80665
3 TEMP_RANGE_C = (-20.0, 80.0)
4 DEFAULT_WINDOW = 3
```

```
1 from sensorkit import config
2
3 config.G_M_PER_S2, config.TEMP_RANGE_C
```

- 한 곳 — 상수는 config 모듈에 집중
- 이름 — 대문자 표기, 단위를 이름에 명시
- 환경별 값 — 경로와 키는 환경변수로 주입
- 금지 — 함수 안에 숫자를 직접 넣기



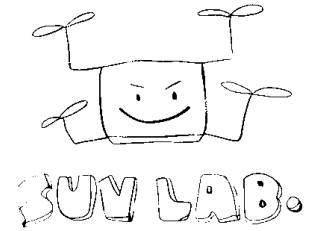
연습문제 2

아래 sensorkit 패키지의 `moving_average` 함수를 서로 다른 세 경로로 import 해 a, b, c 에 담으시오.

```
1 # 준비된 패키지 (src/ 는 sys.path 에 있다)
2 sensorkit/
3 |—— __init__.py      # 함수 재노출
4 |—— filters/
5 |   |—— __init__.py   # 함수 재노출
6 |   |—— moving_average.py
7       def moving_average(xs, n=3)
```

```
1 # a: 패키지 최상위 sensorkit
2 # b: 서브패키지 sensorkit.filters
3 # c: 모듈 ...filters.moving_average
4 # IMPLEMENT HERE
5
6 a is b is c
```

기대 출력 — `True`. 세 경로가 모두 같은 함수 객체를 가리키면 참이 된다



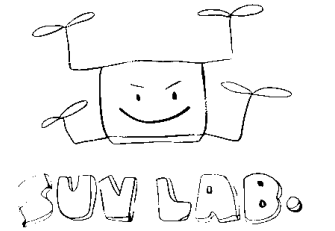
pyproject.toml 이란

패키지의 신분증이자 빌드 설명서

```
1 [build-system]
2 requires = ["setuptools>=68"]
3 build-backend = "setuptools.build_meta"
4
5 [project]
6 name = "sensorkit"
7 version = "0.1.0"
8 requires-python = ">=3.10"
9
10 [project.scripts]
11 sensorkit = "sensorkit.cli:main"
```

- **표준** — PEP 621, setup.py 대체
- **build-system** — 무엇으로 빌드할지
- **project** — 이름, 버전, 의존성, 요구 버전
- **scripts** — 설치 시 만들어질 명령

이름 규칙 — PyPI 이름은 전역, 흔한 단어는 이미 선점



로컬 설치와 빌드

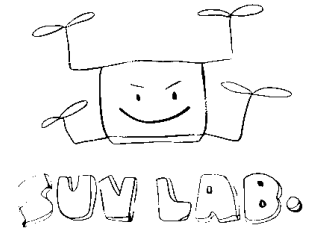
개발 중

- **pip install -e .** — 편집 가능 설치
- **효과** — 소스 수정 즉시 반영
- **확인** — 다른 디렉터리에서 import 시도

배포할 때

- **python -m build** — sdist 와 wheel 생성
- **sdist** — 소스 묶음(.tar.gz)
- **wheel** — 설치용 미리 빌드본(.whl)
- **twine upload** — TestPyPI 에 선행 업로드

버전 — 시맨틱 버저닝, 호환 깨짐 MAJOR, 기능 추가 MINOR, 수정 PATCH



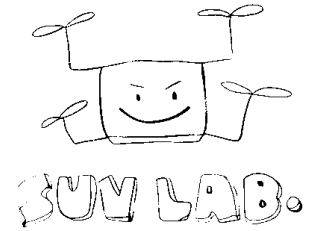
테스트 배치

테스트는 소스 밖에, 설치된 패키지를 대상으로

```
1 # tests/test_filters.py
2 from sensorkit import moving_average as ma
3
4
5 def test_moving_average():
6     assert ma([1, 2, 3], 2) == [1.5, 2.5]
```

```
1 from sensorkit import moving_average
2
3 assert moving_average([1, 2, 3], 2) == [1.5, 2.5]
4 assert moving_average([1], 3) == []
5 print("2 passed")
```

- 위치 — 프로젝트 루트의 tests/
- 이름 — 파일은 test_, 함수도 test_ 로 시작
- 실행 — 루트에서 pytest 한 줄
- import — 상대 경로가 아닌 설치된 이름으로

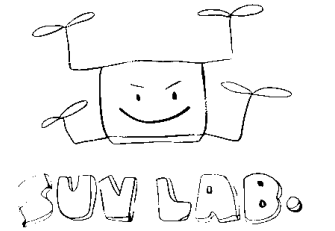


연습문제 3

CLI 진입점 `main` 을 완성하시오. `argv[0]` 을 CSV 경로로 받아 `temp` 열의 평균을 `log.csv: mean=37.22` 형식으로 출력하고, 종료 코드 `0` 을 반환한다.

```
1 import sensorkit as sk
2
3
4 def main(argv=None):
5     # IMPLEMENT HERE
6     ...
7
8 main(["log.csv"])
```

쓸 수 있는 함수 — `sk.load_csv(path)` 행 목록, `sk.column(rows, "temp")` 실수 목록, `sk.mean(xs)` 평균 · **log.csv 의 temp** — 21.5, 21.6, 99.9, 21.4, 21.7 · **기대 출력** — `log.csv: mean=37.22` 와 `0` 두 줄

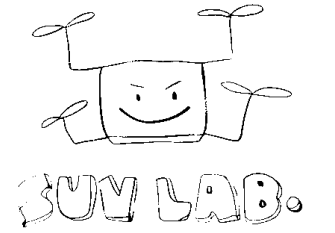


명령으로 등록하기

설치하면 생기는 터미널 명령

```
1 # 터미널에서 쓰는 두 가지 방법
2 $ pip install -e .
3 $ sensorkit log.csv          # 등록된 명령
4 log.csv: mean=37.22
5 $ python -m sensorkit log.csv # __main__.py 경우
6 log.csv: mean=37.22
7 $ echo $?
8 0
```

- **명령 등록** — [project.scripts] 에 sensorkit = "sensorkit.cli:main"
- **python -m** — __main__.py 가 있으면 설치 없이도 실행
- **인자** — 명령 뒤 값은 sys.argv 로 전달
- **종료 코드** — main() 반환값이 셸의 \$? 가 됨



흔한 실패 사례

이렇게 깨진다

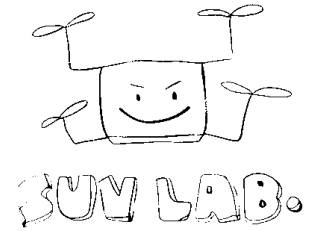
- 순환 **import** — a 가 b 를, b 가 a 를 import
- 이름 충돌 — 파일명이 표준 모듈과 동일
- 경로 의존 — 실행 위치가 바뀌면 파일 탐색 실패
- 무거운 **__init__** — import 만 해도 몇 초

이렇게 푼다

- 공통 모듈 — 양쪽이 쓰는 코드를 하위로 이동
- 이름 확인 — random.py, json.py 같은 이름 금지
- 경로 기준 — Path(__file__).parent 사용
- 지연 **import** — 함수 안에서 필요할 때 import

진단 — sys.path 와 sys.modules 두 곳이면 대개 원인 파악

정리



구조

- **src layout** — 설치해야 import 되게
- **__init__.py** — 공개 API 만 노출
- **역할별 모듈** — io, clean, filters, stats

품질

- **docstring** — 계약을 코드에
- **설정 상수** — config.py 한 곳에
- **테스트** — tests/ 에 assert 로

배포

- **pyproject.toml** — 이름, 버전, 의존성
- **pip install -e .** — 개발 중 설치
- **build, twine** — 배포 산출물